Les fonctions en Python

- Définition et appel d'une fonction
  - Fonction sans sortie
  - Fonction avec sortie
- Portée des variables
- Appel par valeur / par référence

Coordinatrice : Cherifa BEN KHELIL
cherifa.ben-khelil@efrei.fr
Support du cours: Asma Gabis

P1, P1-BN, P1-P, P1-BDX – 2024/2025

01

Définition & Appel d’une fonction

01

Les fonctions

Rappel

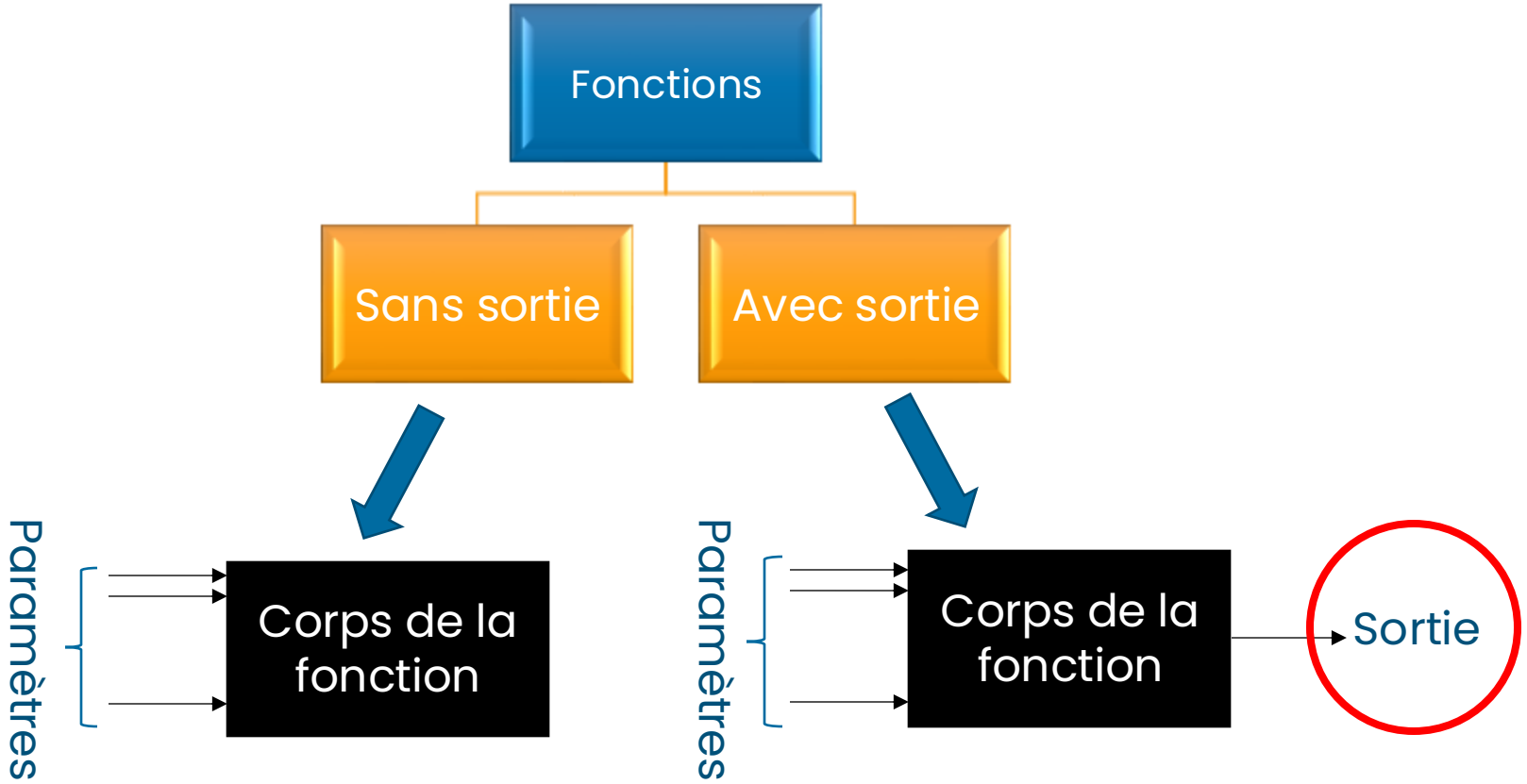
- Les fonctions en Python ont le même rôle que celui décrit dans le cours d'algorithmique

- Une fonction est un sous-programme permettant de réaliser une tâche bien définie.

- **Fonctionnement** :
  - Avant d'utiliser une fonction , il faut d'abord **la définir** ;
  - Au moment de l'utilisation, il faut effectuer **un appel** de la fonction

Les fonctions

Types de fonctions



Les fonctions

Définition d'une fonction sans sortie : Syntaxe

```
def nom_fonction ( Liste des paramètres) :
    Corps de la fonction
```

- **def** : mot clé à désignant le début de la définition d'une fonction
- **nom_fonction** : choisi par l'utilisateur. Il est recommandé de choisir des noms significatifs.
- **Liste des paramètres** : informations données à la fonction, mises entre parenthèses
- **:** Obligatoire après les paramètres
- **def nom_fonction (Liste des paramètres) :** prototype ou signature de la fonction
- **Corps de la fonction**: bloc d'instructions de la fonction, doit être indenté

Les fonctions

Définition d'une fonction sans sortie

Exemples

```
# Fonction sans sortie à 2 paramètres
def add_numbers (a, b):
    res = a + b
    print("{} + {} = {}".format(a, b, res))
```

```
# Fonction sans sortie sans paramètres
def add_numbers ():
    a = int(input("Saisir un entier : "))
    b = int(input("Saisir un autre entier : "))
    res = a + b
    print("{} + {} = {}".format(a, b, res))
```

Les fonctions

Appel d'une fonction sans sortie : Syntaxe

Il est possible d'appeler une fonction n'importe où **après** avoir quitté la fonction ✓

MAIS Il n'est pas possible d'appeler une fonction **avant** sa définition ✗

Définition d'une fonction

```
def nom_fonction ( param1, param2, …):
    Corps de la fonction
```

Appel de la fonction

```
nom_fonction ( arg1, arg2, …)
```

- Les arguments doivent être du même type que les paramètres
- L'ordre sémantique des arguments doit respecter l'ordre des paramètres.

Les fonctions

Appel d'une fonction sans sortie :

Exemples

```
# Fonction sans sortie à 2 paramètres
def add_numbers (a, b):
    res = a + b
    print("{} + {} = {}".format(a, b, res))

# Appel d'une fonction sans sortie avec 2 arguments
add_numbers(1, 3)
```

```
1 + 3 = 4
```

```
# Fonction sans sortie sans paramètres
def add_numbers ():
    a = int(input("Saisir un entier : "))
    b = int(input("Saisir un autre entier : "))
    res = a + b
    print("{} + {} = {}".format(a, b, res))

# Appel d'une fonction sans sortie et sans arguments
add_numbers()
```

```
Saisir un entier : 1
Saisir un autre entier : 3
1 + 3 = 4
```

Les fonctions

Définition d'une fonction avec sortie : Syntaxe

```
def nom_fonction ( Liste des paramètres) :
    Corps de la fonction
    return resultat
```

Mot clé obligatoire pour retourner le résultat de la fonction

Les fonctions

Définition d'une fonction avec sortie :

Exemples

```
# Fonction avec sortie à 2 paramètres
def add_numbers (a, b):
    res = a + b
    return res
```

```
# Fonction avec sortie sans paramètres
def add_numbers ():
    a = int(input("Saisir un entier : "))
    b = int(input("Saisir un autre entier : "))
    res = a + b
    return res
```

Les fonctions

Appel d'une fonction avec sortie : Syntaxe

Définition d'une fonction

```
def nom_fonction ( param1, param2, …):
    Corps de la fonction
    return result
```

Appel de la fonction

```
res = nom_fonction ( arg1, arg2, …)
```

Le résultat retourné par la fonction doit être accueilli dans une variable à l'appel

Les fonctions

Appel d'une fonction avec sortie :

Exemples

```
# Fonction avec sortie sans paramètres
def add_numbers ():
    a = int(input("Saisir un entier : "))
    b = int(input("Saisir un autre entier : "))
    res = a + b
    return res

# Appel d'une fonction avec sortie sans arguments
val = add_numbers()
print("Le résultat de la somme est : {}".format(val))
```

```
Saisir un entier : 1
Saisir un autre entier : 3
Le résultat de la somme est : 4
```

```
# Fonction avec sortie à 2 paramètres
def add_numbers (a, b):
    res = a + b
    return res

# Appel d'une fonction avec sortie à 2 arguments
x = int(input("Saisir un entier : "))
y = int(input("Saisir un autre entier : "))
val = add_numbers(x, y)
print("{} + {} = {}".format(x, y, val))
```

```
Saisir un entier : 1
Saisir un autre entier : 3
1 + 3 = 4
```

Les fonctions

Appel d'une fonction avec sortie dans un script :

Les fonctions ayant une sortie (présence de l'instruction **return**) peuvent être appelée directement au moment de leur utilisation sans passer par une variable intermédiaire qui reçoit le résultat.

Exemples

```
# Fonction avec sortie à 2 paramètres
def add_numbers (a, b):
    res = a + b
    return res

# Cas d'un appel d'une fonction dans le print
x = int(input("Saisir un entier : "))
y = int(input("Saisir un autre entier : "))
print("{} + {} = {}".format(x, y, add_numbers(x, y)))
```

```
Saisir un entier : 1
Saisir un autre entier : 3
1 + 3 = 4
```

```
# Fonction avec sortie à 2 paramètres
def add_numbers (a, b):
    res = a + b
    return res

# Cas d'un appel d'une fonction dans une instruction
x = int(input("Saisir un entier : "))
y = int(input("Saisir un autre entier : "))
carre = add_numbers(x, y) * add_numbers(x, y)
print("({} + {}) * ({} + {}) = {}".format(x, y, x, y,
carre))
```

```
Saisir un entier : 1
Saisir un autre entier : 3
(1 + 3) * (1 + 3) = 16
```

Les fonctions

Appel d'une fonction quelconque à partir d'une autre fonction :

Une fonction peut être appelée aussi à partir d'une autre fonction.

Exemple

```
def min2nb(a, b):
    minimum = a
    if b < a :
        minimum = b
    return minimum
```

Définition de la fonction 1

```
def min3nb(a, b, c):
    return min2nb( min2nb(a, b) , c)
```

Définition de la fonction 2 qui appelle la fonction 1

```
# Appel de la fonction min3nb
x = int(input("Saisir un entier : "))
y = int(input("Saisir un autre entier : "))
z = int(input("Saisir un autre entier : "))
print("La plus petite valeur saisie est : {}".format(min3nb(x, y, z)))
```

Appel de la fonction 2

```
Saisir un entier : 3
Saisir un autre entier : 15
Saisir un autre entier : -1
La plus petite valeur saisie est : -1
```

Les fonctions

Les sorties multiples:

- Généralement, une fonction ne peut retourner qu'une seule sortie.
- En **Python**, il est possible de retourner plusieurs sorties

Et c'est UNIQUEMENT en Python !!!

Exemple

```
def min_max(a, b):
    minimum , maximum = a, b
    if b < a :
        minimum, maximum = b, a
    return minimum , maximum
```

```
Saisir un entier : 5
Saisir un autre entier : 7
La valeur minimum est 5 et la valeur maximum est 7
```

```
# Appel de la fonction min_max
x = int(input("Saisir un entier : "))
y = int(input("Saisir un autre entier : "))
mini, maxi = min_max(x, y)
print("La valeur minimum est {} et la valeur maximum est {} ".format(mini, maxi))
```

Les fonctions

Que se passe-t-il à l'appel d'une fonction ?

Lorsqu'un programme P appelle une fonction F:

- l'exécution de P est suspendue
- L'exécution de F est entamée
- A la fin de l'exécution de F, l'exécution de P reprend à l'endroit où elle a été suspendue.

Exemple

Exécuter le programme suivant sur Python Tutor

```
def min_liste(l):
    mini = l[0]
    for val in l :
        if val < mini :
            mini = val
    return mini

# Appel de la fonction min_liste
L = [12, 6, -1, 56, 21]
m = min_liste(L)
print("La plus petite valeur de la liste est {}
".format(m))
```

Les fonctions

Les variables spéciales

- Le langage Python possède des **variables spéciales** qui sont automatiquement remplies lors de l'exécution d'un programme.
- La variable `__name__` est l'une de ces variables spéciales
- `__main__` est le nom de l'environnement d'exécution principal.
- Le test `__name__ = '__main__'` est couramment utilisé pour vérifier dans quel module (fichier) l'on se trouve lors de l'exécution.
- Si `__name__` est égal à un autre nom, cela signifie que le programme est exécuté à partir d'un module (généralement c'est le nom du fichier aussi)
- Ce test est ajouté pour développer un module pouvant à la fois être exécuté directement mais aussi être importé par un autre module pour apporter ses fonctions.

Les fonctions

Structure générale d'un programme Python :

Finalement, un programme Python type a la structure ci-contre :

```
#####################################
#####  Programme Python type ########
#####  Auteur : .....        ########
#####  Version : .....       ########
#####################################

#####################################
# importation des fonctions externes

from math import sqrt

#####################################
# Déclaration des fonctions utilisateur

def racine (x):
    res = sqrt(x)
    return res

#####################################
# Corps du programme principal

if __name__ == '__main__':
    print("Saisir une valeur :")
    val = int(input())
    racine_val = racine(val)
    print("La racine de {} est {}".format(val, racine_val))
```

Les fonctions

Structure générale d'un programme Python :

Pour une meilleure organisation, il est également possible de regrouper les fonctions par thèmes dans des modules (fichiers) séparés, et les importer pour les utiliser dans le programme principal.

Main.py

```
#####################################
# importation des fonctions externes

from math import sqrt
from monfichier import *

#####################################
# Corps du programme principal

if __name__ == '__main__':
    print("Saisir une valeur :")
    val = int(input())
    racine_val = racine(val)
    print("La racine de {} est {}".format(val, racine_val))
```

monfichier.py

```
#####################################
# Déclaration des fonctions utilisateur

def racine (x):
    res = sqrt(x)
    return res
```

02

La portée des variables

02

La portée des variables

Les variables et les fonctions

Lorsqu'une fonction est définie, ses paramètres ainsi que toutes les variables qu'elle utilise sont locaux.

Analysons l'exemple 1 suivant :

```
# Initialisation d'une variable a
a = 5

# Définition d'une fonction après la déclaration de a
def print_a():
    print(" a = ", a)

# 1er appel de la fonction
print_a()

# Modification de la valeur de a
a = 8

# 2ème appel de la fonction
print_a()
```

```
a =  5
a =  8
```

La variable a est globale ici ➔ La fonction définie l'a bien reconnue et a pu lire sa valeur

La portée des variables

Les variables et les fonctions

Essayons de modifier la valeur de a dans la fonction print_a().

Exemple 2:

```
# Initialisation d'une variable a
a = 5

# Définition d'une fonction après la déclaration de a
def print_a():
    a = a * 2
    print(" a = ", a)

# Appel de la fonction
print_a()
```

A la modification, la fonction considère la variable « a » comme locale mais qui n'a pas été initialisé avant ➔ Erreur

Comment faire alors ?

```
    a = a * 2
UnboundLocalError: local variable 'a' referenced before assignment
```

La portée des variables

Le mot clé « global »

Pour que la fonction puisse modifier la variable, il faut utiliser le mot clé « **global** » devant la variable « a »

```
# Initialisation d'une variable a
a = 5

# Définition d'une fonction après la déclaration de a
def print_a():
    global a
    a = a * 2
    print(" a = ", a)

# Appel de la fonction
print_a()
```

Grâce au mot clé « global », la fonction comprend que »a » est une variable globale ➔ Elle récupère donc sa valeur et parvient à la modifier

```
a =  10
```

La portée des variables

Les variables globales en python

En résumé :

- Les variables globales se déclarent en Python comme une variable locale mais à l'extérieur d'une fonction.

- Pour utiliser une variable globale sans modifier sa valeur, il suffit de l'appeler dans une fonction comme c'est le cas pour une variable locale.

- Si la fonction doit modifier la valeur de la variable, alors il faut la redéclarer en la précédant du mot-clé **global**.

La portée des variables

Les variables globales en python

	Avantage	Inconvénients
Variable globale	Peut être utilisée partout dans un programme sans être transmise préalablement en paramètres de fonction	• L'utilisation de ces variables rend plus difficile la compréhension d'un programme ainsi que son débuggage et sa modification ultérieure. • L'utilisation de variables globales est dangereuse car une simple modification de la variable par une instruction impacte l'ensemble des fonctions qui utilisent cette même variable.

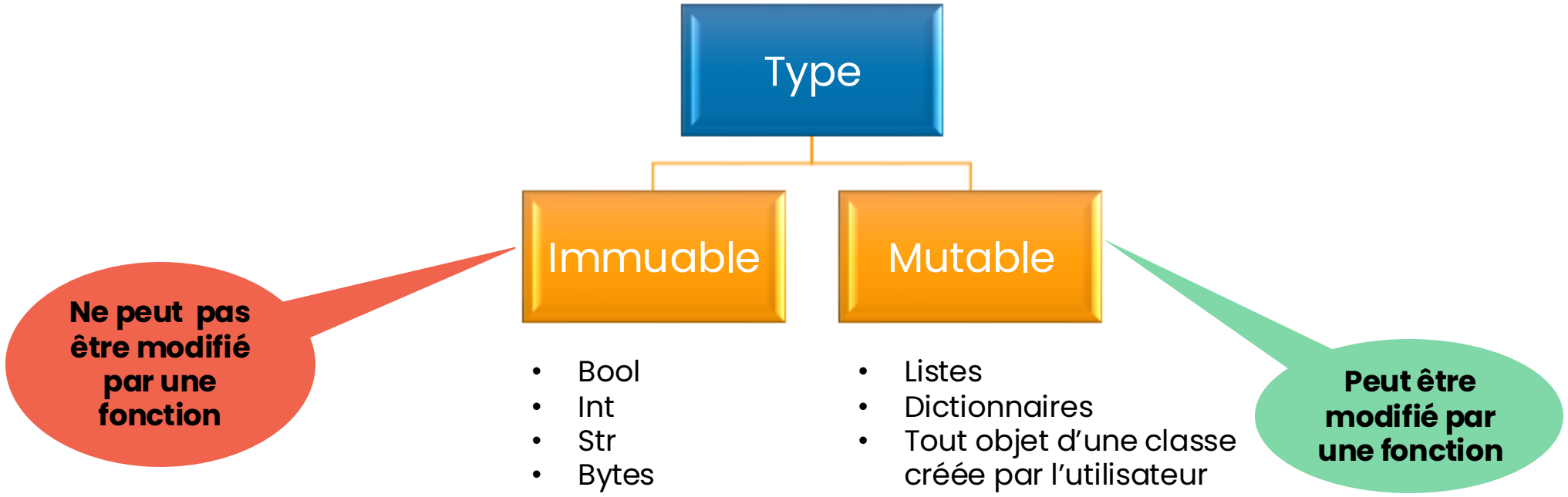
03

Appel par valeur / référence

03

Appel par valeur / par référence

Les types mutables / immuables



Appel par valeur / par référence

Quel est l'impact sur les appels de fonctions ?

Fonction qui ne modifie pas la valeur des paramètres

- Le passage des paramètres est effectué par valeur
- Les valeurs des arguments sont copiées dans les paramètres de la fonction

```
def somme_liste(l):
    somme = 0
    for val in l :
        somme += val
    return somme

# Appel de la fonction
L = [12, 45, 8, 34]
print("La somme des valeurs de la liste est {}".format(somme_liste(L)))
```

Ici, il n'est pas question de modification des valeurs de la liste L

```
La somme des valeurs de la liste est 99
```

Appel par valeur / par référence

Quel est l'impact sur les appels de fonctions ?

Fonction qui essaie de modifier les valeurs des paramètres

- Le passage des paramètres ici doit s'effectuer par **référence** ➔ donner l'adresse et non pas la valeur

- Si le paramètre est de type mutable
    - En donnant son adresse (sa référence), la fonction pourra modifier ses valeurs MAIS pas sa référence

- Si le paramètre est de type immuable
    - Même en donnant son adresse (sa référence), la fonction ne pourra modifier ni ses valeurs ni sa référence

Appel par valeur / par référence

Cas d'un paramètre de type mutable : list

```
# Définition de la fonction
def ajouter_liste(l, val):
    l.append(val)

# Programme principal
if __name__ == '__main__':
    # Initialisation d'une liste L
    L = [1, 2, 3]
    # Affichage des valeurs de la liste avant l'appel de la fonction
    print(L)
    # Appel de la fonction
    ajouter_liste(L, 4)
    # Affichage des valeurs de la liste après l'appel de la fonction
    print(L)
```

Explication:
Le type **list** est mutable ➔ la fonction a donc pu modifier ses valeurs.

```
[1, 2, 3]
[1, 2, 3, 4]
```

Appel par valeur / par référence

Cas d'un paramètre de type mutable : list

```
def modifier_ref_liste(l):
    l = [4, 5, 6, 7]

# Programme principal
if __name__ == '__main__':

    # Initialisation d'une liste
    L = [1, 2, 3]

    # Affichage de la liste avant l'appel de la fonction
    print(L)

    # Appel de la fonction
    modifier_ref_liste(L)

    # Affichage de la liste après l'appel de la fonction
    print(L)
```

Explication:
Même si le type **list** est mutable
➔ la fonction ne peut pas modifier sa référence.

☹

```
[1, 2, 3]
[1, 2, 3]
```

Appel par valeur / par référence

Cas d'un paramètre de type immuable : int

```
def permuter(a, b):
    a, b = b, a

# Programme principal
if __name__ == '__main__':

    # Initialisation de 2 entiers
    x = 4
    y = 5

    # Affichage des valeurs avant permutation
    print("Avant permutation : x = {} , y = {}".format(x, y))

    # Appeler la fonction de permutation
    permuter(x, y )

    # Affichage des valeurs après permutation
    print("Après permutation : x = {} , y = {}".format(x, y))
```

Aucun changement !!

```
Avant permutation : x = 4 , y = 5
Après permutation : x = 4 , y = 5
```

Explication:
Le type **Int** est immutable, donc ne peut être modifié par une fonction externe au programme principal

Appel par valeur / par référence

Cas d'un paramètre de type immuable : str

Erreur !!

```
def modifier_str(s):
    s[0] = 'P'

# Programme principal
if __name__ == '__main__':

    # Initialisation d'une variable de type str
    s = "principal"

    # Affichage de la valeur de s avant l'appel de la fonction
    print("Avant modification : s = {}".format(s))

    # Appeler la fonction de modification
    modifier_str(s)

    # Affichage de la valeur de s après l'appel de la fonction
    print("Après modification : s = {}".format(s))
```

```
    s[0] = 'P'
TypeError: 'str' object does not support item assignment
```

Explication:
Le type **Str** ne supporte pas la modification de ses caractères après une première affectation de valeur.

Appel par valeur / par référence

Cas d'un paramètre de type immuable : str

```
def modifier_str(s):
    s = "fonction"

# Programme principal
if __name__ == '__main__':
    # Initialisation d'une variable de type str
    s = "principal"

    # Affichage de la valeur de s avant l'appel de la fonction
    print("Avant modification : s = {}".format(s))

    # Appeler la fonction de modification
    modifier_str(s)

    # Affichage de la valeur de s après l'appel de la fonction
    print("Après modification : s = {}".format(s))
```

Aucun changement !!

```
Avant modification : s = principal
Après modification : s = principal
```

Explication:
Le type **Str** est immutable, et tout comme la liste, il n'est pas possible de modifier sa référence.

Appel par valeur / par référence

Cas d'un paramètre de type immuable

Comment faire pour contourner cela ?

Faites des propositions …

Appel par valeur / par référence

Cas d'un paramètre de type immuable

Solution 1 : Utilisation des variables globales

Non recommandée !!!

```
def modifier_str():
    global s
    s = "fonction"

# Programme principal
if __name__ == '__main__':

    # Initialisation d'une chaine de caractères
    s = "principal"

    # Affichage de s avant l'appel de la fonction
    print("Avant modification : s = {}".format(s))

    # Appeler la fonction de modification
    modifier_str()

    # Affichage de s après l'appel de la fonction
    print("Après modification : s = {}".format(s))
```

```
Avant modification : s = principal
Après modification : s = fonction
```

Ça a marché !! ☺

Appel par valeur / par référence

Cas d'un paramètre de type immuable

Solution 1 : Utilisation des variables globales

Non recommandée !!!

```
def permuter():
    global x
    global y
    x, y = y, x

# Programme principal
if __name__ == '__main__':
    # Initialisation de deux entiers
    x = 4
    y = 5
    # Affichage des valeurs avant l'appel de la fonction
    print("Avant permutation : x = {}, y = {}".format(x, y))
    # Appeler la fonction de permutation
    permuter()
    # Affichage des valeurs après l'appel de la fonction
    print("Après permutation : x = {}, y = {}".format(x, y))
```

```
Avant permutation : x = 4, y = 5
Après permutation : x = 5, y = 4
```

Ça a marché !! ☺

Appel par valeur / par référence

Cas d'un paramètre de type immuable

Solution 2 : Retourner le résultat

```
def modifier_str(s):
    s = "fonction"
    return s

# Programme principal
if __name__ == '__main__':
    # Initialisation d'une variable de type str
    s = "principal"

    # Affichage de la valeur de s avant l'appel de la fonction
    print("Avant modification : s = {}".format(s))

    # Appeler la fonction de modification
    s = modifier_str(s)

    # Affichage de la valeur de s après l'appel de la fonction
    print("Après modification : s = {}".format(s))
```

```
Avant modification : s = principal
Après modification : s = fonction
```

Ça a marché !! ☺

Appel par valeur / par référence

Cas d'un paramètre de type immuable

Solution 2 : Retourner le résultat

```
def permuter(a, b):
    a, b = b, a
    return a, b

# Programme principal
if __name__ == '__main__':
    # Initialisation de deux entiers
    x = 4
    y = 5

    # Affichage des valeurs avant l'appel de la fonction
    print("Avant permutation : x = {}, y = {}".format(x, y))

    # Appeler la fonction de permutation
    x, y = permuter(x, y)

    # Affichage des valeurs après l'appel de la fonction
    print("Après permutation : x = {}, y = {}".format(x, y))
```

```
Avant permutation : x = 4, y = 5
Après permutation : x = 5, y = 4
```

Ça a marché !! ☺

Appel par valeur / par référence

Paramètres d'une fonction: valeurs par défaut

Python permet aux paramètres des fonctions d'avoir une valeur par défaut, si la fonction est appelée sans l'argument il a la valeur par défaut :

- Il est possible de définir un argument par défaut pour chacun des paramètres.
- Possibilité d'appeler une fonction avec une partie seulement des arguments.
- Si deux arguments fournis, la valeur par défaut pour le deuxième est ignorée.

Appel par valeur / par référence

Paramètres d'une fonction: valeurs par défaut

Exemple 1 :

```
def politesse(nom, titre = "Monsieur"):
    print("Veuillez agréer, {} {}, mes salutations distinguées.".format(titre, nom))

# Programme principal
if __name__ == '__main__':
    # Appel de la fonction politesse avec seulement un argument
    politesse("Dupont")
```

```
Veuillez agréer, Monsieur Dupont, mes salutations distinguées.
```

Le titre « Monsieur » a été ajouté automatiquement

Appel par valeur / par référence

Paramètres d'une fonction: valeurs par défaut

Exemple 2 :

```
def politesse(nom, titre = "Monsieur"):
    print("Veuillez agréer, {} {}, mes salutations distinguées.".format(titre, nom))

# Programme principal
if __name__ == '__main__':
    # Appel de la fonction politesse avec 2 arguments
    politesse("Dupont", "Madame")
```

```
Veuillez agréer, Madame Dupont, mes salutations distinguées.
```

Le titre donné à l'appel est plus prioritaire que le titre par défaut